EEL 5764: Computer Architecture

Lecture 6: gem5 Architectural Simulator

Sandip Ray
Warren B. Nelms Endowed Professor
Department of Electrical & Computer Engineering, University of Florida

Why Architectural Simulation? Moving Beyond Silicon

THE LIMITS OF SILICON & EQUATIONS

- **Silicon Prototyping is Rigid:** Fabricating a physical computer chip on TSMC 3nm/2nm process nodes costs hundreds of millions of dollars and months of lithography tool setup. Once silicon is printed, you cannot alter pipeline depth or cache sizes.
- **Silicon is a Black Box:** Physical processors provide limited internal observability. You cannot easily pause the system clock cycle-by-cycle to inspect transient register states, queue allocations, or physical wire delays.
- **Analytical Models Lack Dynamics:** Pen-and-paper equations (such as our CPU Performance Equation) are valuable but fail to model complex hardware-software dynamic interactions, cache misses, and pipeline resource conflicts.

THE SIMULATION REVOLUTION

- **Complete Design Exploration:** Architectural simulators are software models of physical hardware. They let microarchitects alter cache associativity, branch predictor sizes, or execution ports in minutes by changing variables.
- **Cycle-Accurate Telemetry:** Simulation monitors and logs every event at the nanosecond scale, capturing queue back-pressures, cache miss penalties, and pipeline stalls with complete observability.
- **Pre-Silicon Verification:** Simulators allow software and hardware co-designers to compile and run complex OS-kernels and software benchmarks years before the physical processor core is taped out.

The Core Components & Models of gem5

CPU EXECUTION MODELS

- **AtomicSimple CPU:** A single-cycle, non-pipelined functional model. Excellent for fast functional checks, compiler debugging, and booting an operating system quickly before starting detail runs.
- **TimingSimple CPU:** An in-order model that stalls on memory accesses. It models cache-miss latencies and memory-wall barriers, but executes instruction stream linearly without speculative execution.
- **O3CPU Model (Detailed):** A cycle-accurate, highly detailed out-of-order execution model. It models register renaming, speculative branch resolution, and instruction queues to analyze advanced pipeline bottlenecks.

MEMORY SUBSYSTEM MODELS

- **Classic Cache Model:** The default, lightweight cache simulation module. It implements straightforward, blocking single-core and multicore cache hierarchies. Ideal for quick general studies.
- **Ruby Coherence Engine:** A highly realistic, cycle-accurate memory subsystem. Ruby models exact cache line state transitions, interconnection network topologies, and advanced directory coherence protocols (MESI/MOESI).
- **Interconnection Networks:** Simulates physical network-on-chip (NoC) layouts (crossbars, meshes) connecting processor tiles, caches, and memory interfaces with realistic physical wire routing.

The Dual Execution Modes: SE vs. FS

SYSCALL EMULATION (SE) MODE

- **Host Kernel Translation:** gem5 intercepts program system calls (like printf, open, malloc) and translates them directly into system calls executed by the host Linux operating system kernel.
- **Bypasses OS Overhead:** Does not boot an operating system. This eliminates scheduler noise and driver clock cycles, providing a pristine, clean environment to evaluate standalone workloads.
- **Focus of Homework 1:** Ideal for compiling a single benchmark, launching it on the simulated processor, and measuring execution ticks and instruction-level CPI without background interference.

FULL SYSTEM (FS) MODE

- **Booting a Real OS-Kernel:** Models physical registers, disk controllers, keyboards, and timers, allowing you to boot a complete Linux or Android operating system on top of your simulated processor.
- **Captures System Realism:** Simulates operating system thread scheduling, virtual memory TLB page-table walks, peripheral device interrupt latency, and I/O driver context switches.
- **Heavy Simulation Tax:** Since it must simulate booting and background daemon processes, runs take significantly more time and system resources to complete compared to Syscall Emulation.

Navigating gem5 Output: Configs vs. Stats

THE HARDWARE RECORD: CONFIG.INI

- **Detailed Structural Map:** The config.ini and config.json files record every simulated object and its precise variables (e.g., L1 cache line size, processor clock period, instruction queue size) loaded at start-up.
- **The Verification Audit:** Crucial for verifying that your command-line arguments (like changing cache size or clock frequency) were parsed and registered correctly by the simulation engine.
- **Debugging Interface:** Use this file to trace the object hierarchy: see exactly which cache bank connects to which core's ports on the simulated memory bus.

THE PERFORMANCE TARGET: STATS.TXT

- **The Quantitative Metrics Log:** This text file contains every cycle-level hardware metric gathered during the run. You will spend your homework analysis searching and parsing this file.
- **sim_ticks Metric:** Total simulated run-time of the executable in picoseconds (used to calculate final speedups).
- **system.cpu.ipc / cpi:** Average instructions executed per clock cycle (IPC) and average cycles required per instruction (CPI).
- **system.cpu.numCycles:** Total simulated processor clock cycles spent processing the dynamic instruction stream.
- **Subsystem Deep-Dive:** Provides detailed cache and memory stats (e.g., overall_misses, miss_rate) and branch prediction performance (e.g., lookups, condPredicted, condIncorrect).

Resources and Installation Instructions

Official website: https://www.gem5.org/

Installation Guide: https://www.gem5.org/documentation/general_docs/building

Gem5 users email archive: https://www.mail-archive.com/gem5-users@gem5.org/, **This is where you go for help from the community.**

Our installation route:

Anaconda → Dependencies installation → Gem5 download → Build with scons → Examples